The command chain

How every action an AI agent takes on a Vault computer traces back, through signed permissions that only narrow, to a command the verified owner actually spoke or typed.

Prepared by Pablo Peniche **Status** Draft for discussion · not for distribution **Sources** 21, numbered inline

Also the rig · full research report

IN ONE PARAGRAPH

Every agent action traces back to a verified human.

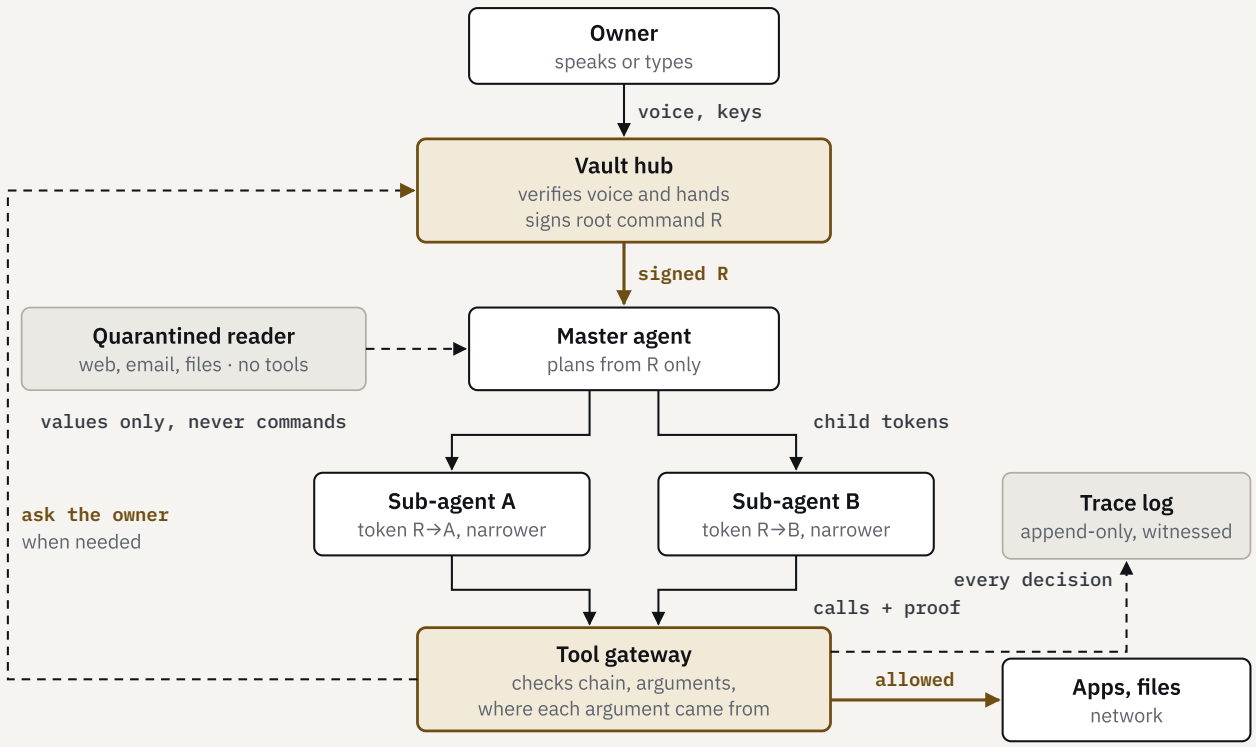
The Vault hub turns each verified human command into a signed root. A master agent plans from verified commands only and hands sub-agents permission tokens that can only narrow. A tool gateway, which is Vault-controlled code rather than an AI model, checks every action against its chain before it runs. It asks the owner when an action is consequential or was shaped by untrusted data, and it writes every allowed, refused and unexplained action to a tamper-evident log. Each piece exists today in some form. No standard and no evaluated system joins them to a continuously verified human[14, 19].

"Proposal" marks design choices made for Vault. Draft standards are labelled as drafts. This page covers the agent side; how Vault verifies the human is in the rig and the full report.

01 · THE CHAIN

From your voice to every tool call

The master agent is the only agent that receives commands, and it only receives commands the hub has signed. Everything below it acts on a narrower copy of that authority. The gateway at the bottom is the only path to tools, files and the network.



From your voice to every tool call (proposal). Only the hub can start a chain, and only after it has verified the voice or hands behind a command. The master agent plans from signed commands; untrusted content goes to a reader that has no tools[1]. Each hop carries a token that can only narrow[2, 3]. The gateway, not any agent, decides what runs. It logs every decision to a witnessed log[9, 11] and sends anything unusual back to you through the hub.

Five rules

01 **Only the hub can start a chain.** A command becomes a root only after the hub has verified the voice or hands that produced it. The root is the hub's signature over the exact transcript, plus the evidence it verified. For consequential commands, the hub reads back the scope the master agent derived ("send Ana the Q3 PDF") before signing, because turning words into permissions is itself a trust boundary[5].

02 **Authority only narrows.** Each sub-agent gets a child token that hashes its parent, is bound to the sub-agent's own key, and can only shrink the allowed tools, arguments, depth and lifetime. Biscuit tokens ship today[3]; UCAN is at release candidate[4]; an IETF individual draft (June 2026) specifies this for agent chains exactly, and leaves the link from the root key to a human as "a deployment responsibility"[2]. That missing link is what the hub provides. A sub-agent cannot act without a minted token, and every minting is logged, so Vault always knows when one agent spawned another, a gap earlier research flagged[13].

03 **Data never becomes a command.** The master agent plans only from signed commands. Web pages, email and files go to a quarantined reader with no tools, which can extract values but cannot change the plan. Consequential arguments (a recipient, an amount, a file to share) must trace back to the human, not to something an agent read. In the CaMeL design this cut successful prompt injections from 300 of 949 to 0 on one model, while solving 77% of tasks against 84% undefended, at about 2.8× the tokens[1]. FIDES with policies cut them from 163 to 1[7].

04 **The gateway checks, not the agents.** Model-level defences keep falling: one 2026 study broke 12 of them, most at over 90% attack success[8]. So the checks run in deterministic code that no prompt can reach. That code verifies four things: that the chain leads back to the hub's key, that the calling agent holds the leaf token's key, that the arguments fit the token, and that consequential arguments came from the human.

05 **Everything is logged, including refusals.** Allowed, refused, escalated and orphan actions (no valid chain) are appended to a Merkle-tree log kept by the hub, the same structure as Certificate Transparency[9, 10]. Checkpoints are countersigned by a witness off the computer, such as the owner's phone[11], so a compromised machine cannot quietly rewrite history. An orphan action is the clearest alarm Vault has.

03 · WHEN YOU GET ASKED

When you get asked

Confirmation prompts work only if they are rare and honest, so the gateway asks only in specific cases (proposal). The text you confirm is generated by gateway code from a structured description of the call[17], not by an AI model. The hub shows or reads it, never the computer's own screen[6].

SITUATION	WHAT HAPPENS
Inside the command's scope, no untrusted data involved	Runs. Logged
Consequential (pay, delete, share outside, install) and inside scope	You confirm the plan once, then it runs step by step without further prompts
Outside the command's scope, or a consequential argument came from untrusted data	Suspended. The hub shows or reads the exact action with a warning such as "recipient came from an email". You re-verify; the hub mints a token for that one call
No valid chain to a verified command	Blocked, logged as an orphan, and you are alerted

There is no general standard for "what you see is what you sign" outside payments. WebAuthn dropped its transaction-text extension and Android deprecated its protected-confirmation screen[20, 21]. That is why the hub's own display and voice are the confirmation channel.

04 · ONE TRACE RECORD

One trace record

Every gateway decision produces one record, signed by the gateway, not by the agent (proposal, adapted from[2, 9, 18]). The log stores hashes; the content stays encrypted on the owner's device and can be revealed selectively.

PART	CONTENTS
Lineage	Record ID; root command ID; parent records (the plan steps and delegations that led here)
Human root	Owner ID; hash of the exact transcript; the hub's verification evidence for that utterance or typing window (channels, scores, times); hub signature
Authority	Token chain from root to leaf; each hop's agent identity (agent ID and signed binary hash); depth; effective permissions; the acting agent's proof of key possession
Action	Tool; hash of the arguments; where each argument came from (human, tool, or untrusted data)
Decision	Policy version; allowed, refused or escalated; if escalated, the confirmation and a hash of the exact text shown
Outcome	Output hash; start and end time on the hub's clock
Log position	Leaf index and the checkpoint current when it was written

What exists and what Vault adds

PIECE	WHAT EXISTS	WHAT VAULT ADDS
A signed human root	AP2 has a non-agentic surface sign the user's payment mandates, for payments only[6]; South et al. propose a human-signed delegation token, as research[5]	A root signed by the hub after continuous voice and hand verification, per command rather than per session
Narrowing tokens	Biscuit (shipping)[3], UCAN (release candidate)[4], attenuating agent tokens (IETF individual draft)[2]	The root key bound to a verified human
Injection resistance	CaMeL and FIDES planners with provenance labels[1, 7]	Enforcement in the gateway, outside any model
Agent protocols	MCP forbids passing tokens through, so command lineage does not travel hop to hop[15]; A2A can escalate a request up a chain of tasks to a human[16]	A chain token carried alongside MCP and A2A calls, checked by the gateway
Tamper-evident logs	Certificate Transparency[9], Sigstore Rekor v2[10], off-device witnesses[11]	Per-action records tied to a human root
Confirmation	Secure Payment Confirmation, for payments only[12]	The hub as the trusted display and voice for every consequential action

Open problems

- **No adaptive-attack test of the planner designs.** CaMeL and FIDES have not yet faced attackers who adapt to them[1, 7, 8].
- **Turning speech into scope.** The master agent's interpretation of a command is a trust boundary. The read-back reduces this risk but adds friction; how often owners actually listen to it is unmeasured[5].
- **Voice as the confirmation channel.** "What you hear is what you sign" has no standard and no security analysis.
- **Agents outside the computer.** Cloud agents only stay in the chain if the services they call check Vault tokens. Until then, their actions are visible only where they pass through the gateway.
- **Utility cost.** About 7 percentage points of task success and about 2.8× tokens for CaMeL-style planning[1].

Sources

1. Debenedetti et al., "Defeating Prompt Injections by Design" (CaMeL, Google DeepMind, 2025). *Research paper.*
2. Niyikiza, "Attenuating Authorization Tokens for Agentic Delegation Chains" (draft-niyikiza-oauth-attenuating-agent-tokens-01, June 2026). *IETF individual draft.*
3. Biscuit token specification. *Shipping open specification.*
4. UCAN specification (v1.0 release candidate). *Open specification, release candidate.*
5. South et al., "Authenticated Delegation and Authorized AI Agents" (2025). *Research paper.*
6. AP2 (Agent Payments Protocol) specification. *Industry specification.*
7. Costa et al., FIDES: information-flow control for AI agents (Microsoft, 2025). *Research paper.*
8. Nasr et al., "The Attacker Moves Second" (USENIX Security 2026). *Peer-reviewed.*
9. RFC 9162, Certificate Transparency Version 2.0. *IETF RFC.*
10. Sigstore, "Rekor v2 is generally available". *Project announcement.*
11. C2SP tlog-witness specification. *Open specification.*
12. W3C Secure Payment Confirmation. *W3C Candidate Recommendation draft.*
13. Chan et al., "Visibility into AI Agents" (FAccT 2024). *Peer-reviewed.*
14. NIST NCCoE, software and AI agent identity and authorization concept paper (Feb 2026). *Government concept paper.*
15. Model Context Protocol specification, authorization (2026-07-28 revision). *Protocol specification.*
16. Agent2Agent (A2A) protocol specification. *Protocol specification.*
17. RFC 9396, OAuth 2.0 Rich Authorization Requests. *IETF RFC.*
18. Nennemann, WIMSE execution context tokens (draft-nennemann-wimse-ect-00). *Expired IETF individual draft.*
19. OpenID Foundation, "Identity Management for Agentic AI" (2025). *Industry whitepaper.*
20. W3C WebAuthn issue #1386, removal of unimplemented extensions including txAuthSimple. *Standards discussion.*
21. Android Open Source Project commit deprecating Protected Confirmation. *Source commit.*
